

UNIVERSIDAD NACIONAL DEL NORDESTE.
FACULTAD DE CIENCIAS EXACTAS Y
NATURALES Y AGRIMENSURA.
CÁTEDRA:
PARADIGMAS Y LENGUAJES DE PROGRAMACION



TRABAJO PRÁCTICO INTEGRADOR.

Grupo 24.

Profesor Responsable: Monzón, Ricardo.

Profesores de Práctico Grupo 1: Mascazzini Matias, Rodríguez Ariel Leandro.

Integrantes: Balenzuela Tobías, De Weert Lautaro, Gómez Ariel, Miño Maria Sol.

Introducción.

El siguiente informe corresponde al desarrollo del trabajo práctico integrador, propuesto por la cátedra de Paradigmas y Lenguajes. El desarrollo del mismo estará dividido en tres fases, la primera comenzará con la argumentación de las decisiones e interpretaciones consideradas dentro del grupo, acompañada con la teoría propuesta por la cátedra. En la segunda fase, la implementación de la librería local-time, donde esta colección de código desarrollado, nos ayuda a resolver y agilizar la resolución del proceso de codificación, veremos cómo son las librerías en Lips, que cambió aquí. En la tercera fase, hablaremos sobre el desafío de la implementación del lenguaje Erlang que conflicto tuvimos, como aprendimos, qué diferencias y similitudes posee con Common Lisps, como la esencia de la lógica sigue igual, pero cambia su sintaxis.

Y por último añadiremos, los errores que tuvimos en el grupo, la resolución de las mismas y la organización.

Aclarado ello, un semaforo es un dispositivo electromagnetico y electronico que ayudan con el control del transito y los peatones. Y su funcion es permitir el paso alternado de los peatonos y vehículos. Segun el manual de conductores de la agencia naciona de seguridad vial, los colores significan:

“LUZ ROJA significa DETENERSE. No cruzar la calle mientras se encuentre encendida la luz roja. Esperar hasta que la luz verde indique la permisión de cruce.

LUZ AMARILLA significa PRECAUCIÓN. Corresponde detenerse y esperar la luz verde para cruzar.

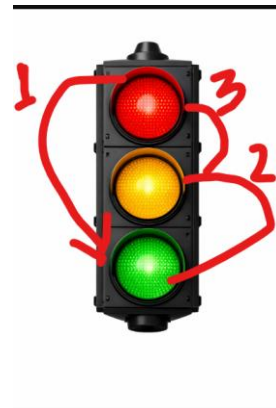
LUZ VERDE significa AVANCE. Es obligatorio hacerlo salvo en caso de falta de espacio al otro lado del cruce.” Agencia Nacional de Seguridad Vial. (2008)

Fundamentos de diseño

FASE 1: RESOLUCIÓN DE REQUISITOS.

Entendiendo la introducción en el requerimiento uno después de los cambios hechos, lo plasmamos en una imagen para ver mejor como seria la transición en la simulación de esos colores. Entonces, el código toma como parámetro el color actual donde está el semáforo y si es posible cambiar a otro color.

Mostrando en pantalla, el resultado dado, esto lo hicimos gracias a la estructura condicional COND, que nos permitió expresar varias condiciones, se evalúa secuencialmente y se detiene obtiene un valor NO-NIL, devolviendo por pantalla el resultado o el mensaje o la lista de ese valor NO-NIL¹. Después dentro de cada condición nos pareció propicio la incorporación del operador lógico **and** quien evalúa que sea T dos condiciones la primera, con el eq compara dos valores, y evalúa si tienen el mismo valor, en este caso si las letras color-actual es igual a en-rojo o en-verde o en-amarillo y en segundo lugar si cambiar-a es igual a verde o amarillo o rojo. Entonces, si las dos evaluaciones son T de alguno de los tres and, devuelve una lista con el mensaje, del color actual y el color a que cambia, si ninguno es T, se imprimirá automáticamente color-actual y acción-por-defecto.



(defun transición (color-actual cambiar-a)

(cond

((and (eq color-actual 'en-rojo)(eq cambiar-a 'verde))(list color-actual 'cambiar-a-verde))

((and (eq color-actual 'en-verde)(eq cambiar-a 'amarillo)) (list color-actual 'cambiar-a-amarillo))

((and (eq color-actual 'en-amarillo)(eq cambiar-a 'rojo))(list color-actual 'cambiar-a-rojo))

(t (list color-actual 'accion-por-defecto))))

En el segundo requerimiento, nuestra función timer, en un principio habíamos implementamos la simulación de un temporizador automático, mediante la entrada por parámetro de tiempo llamado tiempo-unix, que es un formato de tiempo Unix (cantidad de segundos transcurridos) en esta caso en nuestra investigación nos dimos cuenta que comienza con los segundos transcurrido desde 1 de enero de 1970, si bien en la primera fase es posible que en esta etapa

-
- ¹ Monzón, R. (2026). *PyL - 2026 – Parte 3 estructuras condicionales*. [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.

no se aprecie o note la diferencia con claridad como en la segunda fase. Porque, la estrategia central de la función se apoya mediante la división del tiempo-unix con el ciclo completo que propone el proyecto que es 225, empero en la división no queríamos el resultado de este sino el resto y lo logramos con el operador matemático `rem`, porque siempre dará como resultado un valor dentro del ciclo que comprende entre el 0 y el 225 como si fuera un reloj que se actualiza cada 225 seg. Sin importar que tan grande sea los segundos del tiempo-unix.

También realizamos una composición de funciones, con `sumar-ciclo` si bien no tiene parámetros, la suma de esta corresponde al ciclo en segundos de rojo 90 seg.-> amarillo 6 seg.-> verde 120 seg mas los 3 seg. De cada intermitencia y si alguien dentro de la simulación desea cambiar ese ciclo puede hacerlo sin problemas y sin modificar nada en la función `timer`, brindando mantenibilidad.

A partir de este resultado evaluado por un `cond`, evalúa y brinda el mensaje del cual sea un valor `NO-NIL`, en ello es si es un numero negativo dará como respuesta que esos números no se evalúan porque sería un resultado ilógico y así evitamos errores, en segundo lugar, si es menor a 90 corresponde al estado de semáforo en rojo, si es menor a 96 pero mayor a 90 corresponde al semáforo en amarillo y cualquier otro en verde, esto realizamos antes.

```
TIMER
[2]> ;; =====
;; FUNCIÓN: Sumar-ciclos
;; NATURALEZA: Pura.
;; ESTRATEGIA: Devuelve una suma de la cant. de segundos de duración del semf. en rojo 90 seg., amarillo 6 seg. y verde 120 seg.
;; IMPACTO: No destructiva(Auxiliar para función timer y recomendación-ciclo-ing)
;; =====

(defun sumar-ciclos ()
  (+ 90 6 120))

;; =====
;; FUNCIÓN: timer
;; NATURALEZA: Pura (dado un timestamp, siempre retorna el mismo color)
;; ESTRATEGIA: Selectiva (uso de cond)
;; IMPACTO: No destructiva (solo calcula y devuelve resultados)
;; =====

(defun timer (tiempo-unix)
  (cond
    ((< tiempo-unix 0)
     "El tiempo Unix no puede ser negativo")
    ((< (rem tiempo-unix (sumar-ciclos)) 90)
     'en-rojo)
    ((< (rem tiempo-unix (sumar-ciclos)) 96)
     'en-amarillo)
    (t
     'en-verde)))
```

Pero en la parte dos de iteración (cuestion que en un principio entendimos mal), nos comenta que las funciones de esa índole modifican el ciclo como esta en la imagen anterior y agrega un ciclo de 225 más la incorporación de estas funciones, para evaluar en la función `timer`. Aquí recurrimos a dos funciones una recursiva y otra `timer` con composición de funciones.

La primera utiliza recursión que es, una técnica de programación donde la función se llama así misma para resolver un problema dividiendo en pequeños problemas. Se divide en dos partes el caso base que es la condición donde la recursión se detiene y no entra en bucle infinito y el caso recursivo donde la función se llama así misma y los argumentos se modifican hasta llegar al caso base.

```
(defun evaluar-tiempo-en-ciclo (resto-tiempo estado-actual)
```

```

(cond
  (((< resto-tiempo (tiempo-estado estado-actual))
    estado-actual)
   (t
    (evaluar-tiempo-en-ciclo
     (- resto-tiempo (tiempo-estado estado-actual))
     (siguiente-estado estado-actual))))))

```

En el caso base si el tiempo es menor a la duracion del tiempo estado, es decir, menor que 90 es rojo, si es mejor a 93 rojo intermitente si es mejor que 96 es verde y así sucesivamente, solo en caso de ser menor, la recursión frena y devuelve el color de estado-actual.

En el caso recursivo si el tiempo no es menor a la duracion entra en la rama del true que resta lo que ya paso de la luz actual y va comparando, si es rojo ahora pregunta por el verde y si no por el amarillo, hasta que llega y devuelve lo que corresponde.

Ahora bien, La función timer ingresa por parámetro, el tiempo-unix y evalúa mediante un cond dos decisiones si el tiempo-unix es menor a 0 emitirá un mensaje de error, sino ingresa al true e invoca la función recursiva, que le pasa por parámetro el resto de la división de tiempo-unix con la función sumar ciclos que es igual a 225, por otro lado 'rojo este ahí como el parámetro determinado si el tiempo- unix es igual a 0 o menor a 90.

```

(defun timer (tiempo-unix)
  (cond
    (((< tiempo-unix 0) "El tiempo Unix no puede ser negativo")
     (t (evaluar-tiempo-en-ciclo (rem tiempo-unix (sumar-ciclos)) 'rojo)))
  )
)

```

A modo aclaratorio el ciclo del semáforo pose esa duración porque el requerimiento así lo aclaro ya que en realidad, según las recomendaciones de los ingenieros civiles la determinación del tiempo, se debe realizar por medio de un cálculo matemático donde se calcula, la cantidad de vehículos que usan la intersección, con el volumen de tráfico a horas de mayor demanda así evitar congestionamientos y demoras, el tipo de vehículo que circulan si solo autos livianos o también camiones o colectivos que ocupan más espacios porque requieren más tiempo para la aceleración, y para establecer el ciclo del semáforo es necesario determinar bien las fases y los tiempos perdidos en la reacción a los cambios de luces, la velocidad de los vehículos, la

distancia aproximada de frenado y las dimensiones de la intersección porque no él lo mismo una calle, una avenida o una ruta. Esa fórmula matemática se denomina La fórmula de Webster ²

C_o = Tiempo óptimo del ciclo del semáforo (segundos).

L = Tiempo total perdido por ciclo debido a los cambios de fase.

$$C_o = \frac{1,5L + 5}{1 - \sum_{i=1}^{\phi} \beta_i}$$

β_i = Relación entre el flujo vehicular actual y el flujo de saturación del acceso o carril crítico de la fase i .
 Σ = Sumatoria de todos los valores β_i .
 ϕ = Número de fases del semáforo.

Siendo esto muy complejo.

En el requerimiento número tres es una función que nos hizo pensar mucho porque no sabíamos si habría que utilizarla para dar un formato de impresión para los resultados de la primera (transición) y la segunda función(timer) que hicimos para el requerimiento o hacerla de manera particular, al final decidimos hacerla sin la composición de funciones después de debatir también con compañeros de otros grupos, nos dimos cuenta que entonces la función auditoria no calcularía nada por sí misma y solo acomodaría con un formato de impresión las dos otras funciones además estas son funciones puras que no tienen efectos secundarios mientras que auditoria es impura porque al imprimir en la terminal imprime un formato de mensaje siendo impura.

Siendo una conclusión compartida fue que mezclar ambas naturalezas dentro de una sola función dificultaría las pruebas unitarias y reduciría la claridad del código.

Mediante eso la función auditoria es independiente recibe como parámetro el tiempo unix, el color anterior y el color nuevo, conservando una responsabilidad clara. Bueno entonces la función toma como parámetro un tiempo-Unix un color actual y un color cambiar a. Mediate la condición de if que evalúa si que tiempo Unix sea mayor a 0, en clases vimos que if es una estructura condicional que expresa opcionalmente su parte THEN, esto es que se compone de tres partes su evaluación primero la forma a evaluar que es (*if* ($\geq$ tiempo-unix 0) si esta es un a valor T pasa ese valor a THEN lo ejecuta y lo imprime (*format t "Tiempo ~d: la luz ha cambiado de ~a ~a tiempo-unix color-anterior color-nuevo)* sino, ELSE imprimirá *nil*.³

En cuanto a la construcción del mensaje utilizamos *format*⁴ que en conjunto con *~a* nos permito construir un texto fijo, sin comillas y con valores combinados que son tiempo-Unix (el

² Autor desconocido. (s.f.). Cálculo de tiempos en semáforos. Scribd. Recuperado de <https://es.scribd.com/presentation/435479012/calculo-de-tiempos-en-semaforos>

³ Monzón, R. (2026). PyL - 2026 - Funcional Parte 3: Estructuras condicionales, iterativas y entrada/salida [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.

⁴ Monzón, R. (2026). PyL - 2026 - Funcional Parte 4: Asignación, funciones lambda, recursividad y streams [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del

cual es mismo tiempo-Unix explicado con anterioridad) y color-anterior, color-nuevo, que permiten que se imprima tal cual como se ve sin la necesidad de construir una lista o que se imprima entre comillas, la salida de t no indica imprimir en pantalla y % hace alusión a los saltos de líneas.

Como resultado nos imprime Retorna una leyenda en pantalla en la que se especifica, el momento en el que el semáforo realizó el cambio de color, y el paso del color antiguo al nuevo.

```
;=====
; FUNCIÓN: auditoria
; NATURALEZA: Impura (imprime en pantalla)
; ESTRATEGIA: Secuencial (uso de format para construir el mensaje)
; IMPACTO: No destructiva
;=====
(defun auditoria (tiempo-Unix color-anterior color-nuevo)

  (if (>= tiempo-Unix 0)

    (format t "Tiempo ~d: la luz ha cambiado de ~a a ~a"

            tiempo-Unix color-anterior color-nuevo)

    nil)

)
```

Admitimos que, llegados a este punto, uno de los más conflictuados fue el requerimiento 4.a (léase en profundidad la confusión en el apartado de los errores) dice calcular un ciclo completo

“4a. Función duracion-ciclo

- **Entrada:** un número determinado de segundos
- **Propósito:** Calcular duración total de un ciclo completo (rojo → amarillo → verde → rojo)

”

En primer lugar, uno al leer esto se pregunta: “¿Con **un** número determinado de segundos?” Porque si era un solo número no tiene sentido, no calcula nada ya que, si se ingresa por parámetros, como salida daría ese mismo número, en segundo lugar, la forma como está planteado el ciclo es erróneo, ya que en los manuales de vialidad o la recomendación de ingeniería que veremos más adelante, se considera que un ciclo es Rojo → Amarillo → Verde. Al verlo de esta manera, decidimos hacer una función que ingrese por parámetro la duración de los segundos por separados, es decir los segundos del semáforo en Rojo, aparte el Amarillo y por último el Verde para generar una suma, y como salida brindar la duración del ciclo total. En lo que concierne con el nombre de la función hace referencia a lo que pide la consigna

calcular-duración-total y los nombres de las variables que pasan por parámetro es semf-R (semáforo Rojo) sen-A (semaforo Amarillo) y semf-V (semaforo en verde).

```
:: =====  
;; FUNCIÓN: calcular-duracion-total  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: Devuelve la suma de la cant. de segundos que tarda el semaforo en rojo, el  
semáforo en amarillo y el semáforo en verde.  
;; IMPACTO: No destructiva. Función auxiliar para la función recomendación-ciclo-Ing. y  
calcular porcentaje hora.  
;; =====
```

(Defun calcular-duracion-total (sema-R sema-A sema-V)
(+ sema-R sema-A sema-V))

Bien en el requerimiento punto 4.b nosotros comprendimos que esta función evalúa la cantidad de segundos del ciclo-completo de un semáforo, este se compone de la suma de la duración del semáforo en rojo, en amarillo y verde. A partir de esa lógica, dividimos el problema en tres funciones, primero la Duración Ciclo que mediante el uso de un cond evaluamos tres alternativas en relación con el reglamento considerado optimo en general por el gobierno nacional, si bien, los reglamentos en tema duracion en Argentina se rigen dependiendo de cada municipalidad, no tenemos un reglamento escrito en la provincia de corrientes e incluso en la ley de tránsito N.º 24.449. como en la Dirección Nacional de Vialidad no contempla explícitamente la duración, solo normaliza que el semaforo en rojo deben detenerse en la línea marcada en la calle, el semaforo en amarillo advierte el peligro inminente de cruzar en conjunto de varias recomendaciones y el verde si continua con su paso.⁵ Como en la Dirección Nacional de Vialidad las especificaciones del semaforo se basan en su ubicación, altura, color, entre otros que no son de importancia aquí. Entonces, en la búsqueda encontramos, que la duracion del semaforo depende de las recomendaciones de un ingeniero civil, que tiene en cuenta la circulación de peatones, velocidades y tamaño de los vehículos, como están predisuestas las intersecciones conllevando a una idea en común el cual es la duracion de no menos de 35 seg. Ni mayor a 150 seg.⁶⁷, y a diferencia de Bs As quien por su mayor caudal vehicular y personas sus ciclos recomendables son desde 45 seg. Y los 130 seg. Tomamos en consideración las recomendaciones de la ingeniería civil.

⁵ Congreso de la Nación Argentina. (1995). *Ley Nacional de Tránsito N.º 24.449*. Texto actualizado. Recuperado de <https://www.argentina.gob.ar/normativa/nacional/ley-24449-818/actualizacion>

⁶ Autor desconocido. (s.f.). *Cálculo de tiempos en semáforos*. Scribd. Recuperado de <https://es.scribd.com/presentation/435479012/calculo-de-tiempos-en-semaforos>

⁷ Unknown. (2014, abril 21). *Distribución de los tiempos de semáforos (I)*. Todo de Ingeniería Civil. Recuperado de <https://www.ingenieracivil.com/2014/04/distribucion-de-los-tiempos-de.html>

Entonces, retomando lo anterior evaluamos si es menor a 35 lanza mensajes de recomendación por posibles riesgos de colisión, por el escaso tiempo de realizar el frenado correspondiente. Si es menor a 150 tiene como salida un mensaje optimo debido a que está dentro de los parámetros recomendados, pero si es mayor a 150 la condición utilizamos como true al final del cond es el mensaje de posibles daños psicológicos, infracciones de tránsito por la impaciencia de los conductores.

```
;; =====  
;; FUNCIÓN: Evaluar-Ciclo  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: Ingresa como parámetro la cantidad de seg que dura la suma de un ciclo.  
Devuelve un mensaje correspondiente a las distintas opciones relacionadas a la duración de  
segundos del semaforo.  
;; IMPACTO: No destructiva  
;; =====
```

```
(defun Evaluar -ciclo (nro-segundos)  
(cond  
((< nro-segundos 35)  
"Ciclo por debajo del mínimo recomendado por Ingenieros Civiles.Existe riesgo de colision."))
```

```
((<= nro-segundos 150)  
"Ciclo dentro de los parametros recomendados en por la Ingenieros Civiles ")  
t)
```

```
"Ciclo por encima del máximo recomendado por Ingenieros Civiles.Existe riesgo de  
incrementar incumplimiento en las normas de transito"))))
```

Al terminar esto, creamos la función Recomendación-ciclo-Ing., toma como parámetro la duracion de los segundos del semaforo en rojo, aparte el amarillo y el verde. Y Mediante una composición de funciones, primero realiza la suma de los segundos por semaforo y ahí recién evalúa si es recomendable o no, lo hicimos así porque nos pareció optimo cuando el usuario, determina el ciclo optimo mediante los distintos segundos en cada fase del semaforo, es decir, puede pensarlo como “Si se evalúa como semaforo en rojo 35- semaforo en Amarillo 6 y verde en 120” siendo más útil y reutilizable.

```
;; =====  
;; FUNCIÓN: Recomendacion-ciclo-ing  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: Devuelve el mensaje segun la suma de calcular los ciclos, por lo tanto, es  
una composición de la función Evaluar-ciclo y calcular-duracion-total.  
;; IMPACTO: No destructiva.  
;; =====
```

```
(defun recomendacion-ciclo-ing (semf-R semf-A semf-V)
```

```

    (Evaluar-ciclo (calcular-duracion-total semf-R semf-A sem-V)))
  )
)

```

Continuando el análisis el requerimiento cinco la Planificación Temporal, no pedía que calculemos cuantos ciclos se completan en un número determinados de minutos, su nombre ciclos-por-tiempo proviene de la actividad y con entrada ingresa por parámetro la cant-minutos, dentro de esa función, comienza con la división entre la cantidad de minutos multiplicado por 60 seg. (duracion de segundos dentro de minutos) y la divide con la función sumar-ciclos, porque tiene la cantidad de segundos propuesto por los primeros requerimientos. Al ser una división el Lenguaje Lisp muchas veces transforma esa división en una fracción cuestión que para nuestro problema actual no es útil y tiende a la con función.

Por ello la función float fue esencial ya que nos ayuda a pasar de una fracción $\frac{1}{2}$ a un número decimal 0.50 por último ese número decimal es evaluado por la función truncate como aquí queremos saber únicamente los ciclos completos en la determinada cantidad de minutos sin importante los segundos sobrantes del ciclo entonces, esta función fue optima porque, todos los números decimales los vuelve al entero más cercano al cero, es decir, 1,5 se convierte en 1.

```

;; =====
;; FUNCIÓN: Ciclos-por-tiempo
;; NATURALEZA: Pura.
; ESTRATEGIA: Es calcula la cant. de ciclos completados primero, pasa los min a segundo,
luego divide con sumar-ciclos y devuelve el numero anterior.
;; IMPACTO: No destructiva
;; =====

```

```

(defun ciclos-por-tiempo (cant-min)
  (truncate (float (/ (* cant-min 60) (sumar-ciclos)))))

```

En el requerimiento número 6, nosotros pensamos que este nombre era adecuado para que muestre que tiene realiza, calcularPorcentajeHora. Recibe como parametros la duracion en segundos de cada fase del semáforo (rojo, amarillo, verde) y devuelve una lista que asocia cada color con su porcentaje de participación dentro del ciclo. Entonces a partir del ingreso (semf-R, semf-A, semf-V) se arma una lista, que dentro de ella funciona de la siguiente manera en cada color Rojo- Amarillo –Verde se haya se realiza una serie de cálculos que gracias por la composición funcional.

Primero, invoca a calcular-duracion-total pasándole los tres parámetros, obteniendo de esta manera, la duración completa de un ciclo del semáforo sin repetir la lógica de suma en esta función, Luego divide al anterior con 3600 porque representa los segundos de una hora por esa duración total y aplica truncate para obtener la cantidad entera de ciclos completos que ocurren en una hora. Mediante esa cantidad de ciclos, multiplica la duración de cada color para saber cuántos segundos totales ocupa ese color durante la hora completa. Ese resultado se divide entre 3600 y se multiplica por 100 para obtener el porcentaje real de participación de cada color

en una hora de funcionamiento del semáforo, además usamos la función float porque algunas veces las divisiones en lisp dan como resultado fracción cuestión que nos entorpecería en la comprensión de los porcentajes es decir si la división es $\frac{7}{4}$ su decimal es 1.75 mejorando la comprensión y legibilidad.

```
;;=====
;; FUNCIÓN: calculaPorcentajesHora
;; NATURALEZA: Pura
;; ESTRATEGIA: Recibe la duración en segundos de cada fase del semáforo,
;;calcula cuántos ciclos completos entran en una hora mediante composición
;; con calcular-duracion-total, y devuelve el porcentaje real de cada color
;; en 3600 segundos. Usa float para legibilidad del resultado.
;; IMPACTO: No destructiva
;;=====
```

```
(defun calculaPorcentajesHora (sema-R sema-A sema-V)
```

```
(list
```

```
'ROJO (float (* (/ (* sema-R (truncate (/ 3600 (calcular-duracion-total sema-R sema-A sema-V)))) 3600) 100))
```

```
'AMARILLO (float (* (/ (* sema-A (truncate (/ 3600 (calcular-duracion-total sema-R sema-A sema-V)))) 3600) 100))
```

```
'VERDE (float (* (/ (* sema-V (truncate (/ 3600 (calcular-duracion-total sema-R sema-A sema-V)))) 3600) 100))
```

```
)
```

Ahora bien, antes de seguir con las siguientes funciones iterativas hay que hablar cuál es su diferencia con el paradigma funcional, en primer lugar, este último se centra en resolver el problema con el cómo llego al resultado mientras que el iterativo se centra más en el que quiero obtener, dando instrucciones explícitas a la computadora mediante algoritmos que describe detalladamente paso a paso para llegar a la solución, en cambio, el funcional describe el problema mediante funciones matemáticas y de composición funciones y así hasta llegar a lo que deseo.

Otro de las diferencias son las formas en cómo funcionan las estructuras de condiciones mientras en el iterativo utiliza bucles como LOOP, DO, DOTIMES, que se ejecutan continuamente en ciclos a menos que se les indiquen que pare y se evalúan del principio hasta el final, y el funcional la recursividad o la elección de distintas opciones se aplican con IF, COND, MAPCAR O RECURSION.

En Lisp se puede representar ambas porque, se considera un lenguaje **multiparadigma** porque también soporta estructuras iterativas e imperativas.

Una vez aclarado esta diferencia, continuamos con nuestra explicación,

En la Extensión 1 de intermitencia de seguridad se solicita agregar unos 3 segundos en los cambios de las luces, en ello hicimos tres funciones

Esta función controla la transición entre los distintos estados y permite que el semáforo siga su ciclo de manera automática.

El primero recibe como parámetro el estado actual que es lo mismo que el color que ingresa puede ser rojo, rojo-intermitente, amarillo amarillo-intermitente, verde y verde-intermitente a partir de eso analiza paso por paso cual es el estado y devuelve el estado que sigue en la secuencia, por ejemplo: Si recibe rojo, devuelve rojo-intermitente; Si recibe verde, devuelve verde-intermitente; Si recibe amarillo-intermitente, vuelve a rojo.

```
:: =====  
;; FUNCIÓN: de transición de estados  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: pregunta ¿Cuál es el próximo estado del semáforo?, entonces recibe un  
estado y devuelve el siguiente.  
;; IMPACTO: No destructiva  
;; =====
```

```
(defun siguiente-estado (estado)  
  (cond  
    ((equal estado 'rojo) 'rojo-intermitente)  
    ((equal estado 'rojo-intermitente) 'verde)  
  
    ((equal estado 'verde) 'verde-intermitente)  
    ((equal estado 'verde-intermitente) 'amarillo)  
  
    ((equal estado 'amarillo) 'amarillo-intermitente)  
    ((equal estado 'amarillo-intermitente) 'rojo)  
  
    (t 'estado-invalido)))
```

En la función tiempo-estado la función debe indicar cuando dura cada estado, recibe como igualmente por parámetro como la funciona anterior y busca un valor asociado a ese estado y devuelve la cantidad de tiempo que permanece activo, por ejemplo, Permite controlar la duración de cada luz para que el semáforo funcione con tiempos definidos, por ejemplo, el rojo

dura segundos, el verde dura 25 segundos y el amarillo 5 segundos, os estados intermitentes duran 3 segundos.

```
;; =====  
;; FUNCIÓN: tiempo-estado  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: su trabajo es decir ¿Cuántos segundos dura este estado?, Recibe un estado y devuelve un número.  
;; IMPACTO: No destructiva  
;; =====  
(defun tiempo-estado (estado)  
  (cond  
    ((equal estado 'rojo) 90)  
    ((equal estado 'verde) 120)  
    ((equal estado 'amarillo) 6)  
  
    ((equal estado 'rojo-intermitente) 3)  
    ((equal estado 'verde-intermitente) 3)  
    ((equal estado 'amarillo-intermitente) 3)  
  
    (t 0)))
```

Por último, en mostrar estado, presenta al usuario el estado actual del semaforo en forma legible, recibe un estado interno del sistema, lo convierte en un texto descriptivo, y lo devuelve como una cadena de caracteres, entonces en resumen las tres funciones hacen esto

```
;; =====  
;; FUNCIÓN: mostrar colores  
;; NATURALEZA: Pura.  
;; ESTRATEGIA: mostrar información al usuario. Recibe un estado y devuelve un mensaje o texto para mostrar.  
;; IMPACTO: No destructiva  
;; =====  
  
(defun mostrar-estado (estado)  
  (cond  
    ((equal estado 'rojo) "ROJO")  
    ((equal estado 'rojo-intermitente) "ROJO-INTERMITENTE")  
  
    ((equal estado 'verde) "VERDE")  
    ((equal estado 'verde-intermitente) "VERDE-INTERMITENTE")  
  
    ((equal estado 'amarillo) "AMARILLO")
```

```
((equal estado 'amarillo-intermitente) "AMARILLO-INTERMITENTE")
```

```
(t "ESTADO INVALIDO"))))
```

Las tres funciones operan de manera coordinada dentro del sistema de control del semáforo. La función siguiente-estado determina la transición lógica hacia el próximo estado del ciclo; tiempo-estado define la duración específica de cada fase según los parámetros establecidos; y mostrar-estado traduce el resultado en una representación comprensible para el usuario, facilitando la interpretación visual del proceso.

Esta función recibe una colección de datos que contiene los cambios realizados por el semáforo, luego de ello crea un archivo o lo va reemplazando donde se almacenara el informe.

Entonces, después de escribir el encabezado, recorre todos los registros recibidos y guarda cada transición de estado, indicando desde que estado paso el semáforo y hacia cual lo hizo.

Elegimos así porque nos permitió crear un historial de funcionamiento para que el sistema a lo revise posteriormente, aquí manejamos los stream que es un tipo de dato que mantiene la información sobre el fichero que este asociado. Bueno, **:direction :output**: nos ayuda a que se habrá un modelo de salida-escritura, permitiendo enviar los datos desde el programa hasta el archivo, **if-exists :supersede**: es nuestra instrucción de seguridad que si el archivo ya existe lo reemplaza por uno nuevo para que evite duplicaciones o sobreescrituras.

Después en la función utilizamos sobre los datos un mapcar que nos sirve para procesar la cada registro que se ingresa y aplica la función lambda para cada registro ingresado escribiendolo en el formato solicitado en un archivo de texto, y ejecuta la función hasta que no queden más elementos. Dentro del mismo bucle pensamos que para extraer los componentes de cada lista utilicemos **(first registro)**: para extraer el primer elemento, que corresponde a la **fecha y hora** (Timestamp). Después el **(second registro)** extrae el segundo elemento, verbigracia el **estado de origen** (ej. ROJO) y ergo, **(third registro)**, extrae el tercer elemento: el **estado de destino** (ej. VERDE). Con este formato "**~A - Transición: ~A -> ~A~%**", por lo tanto, cada llamado dentro del dolist añade una nueva línea al archivo hasta completar el historial de movimientos del semáforo

Recapitulando esta función transforma una estructura de datos en un **documento de texto** cumpliendo con las reglas de estilo de la cátedra al utilizar una descomposición clara y herramientas de iteración adecuadas como dolist en lugar de recursión manual para tareas de salida de datos.

```
(defun informe (datos)
```

```
(defun informe (datos)
```

```
(with-open-file (stream "informe-ejecucion-semaforo.txt"
```

```
  :direction :output
```

```
  :if-exists :supersede)
```

```
(format stream "Informe de Ejecución del Sistema Semafórico~%")
(format stream "=====%%")
(mapcar #'(lambda (registro)
  (format stream "A - Transición: ~A -> ~A%"
    (first registro)
    (second registro)
    (third registro)))
  datos)
(format stream "~%--- Fin del Informe ---"))
```

Por ejemplo:

Informe de Ejecución del Sistema Semafórico

```
=====
2024-06-04 14:30:15 - Transición: ROJO -> VERDE
2024-06-04 14:30:30 - Transición: VERDE -> AMARILLO
2024-06-04 14:30:35 - Transición: AMARILLO -> ROJO
```

Para finalizar esta primera fase, queremos aclarar sobre el requerimiento número 7 si bien debería ir dentro del código principal, tuvimos una problemática cuando se ejecutaba todo el código junto y si bien, en la consigna nos habla sobre respetar el formato de cómo se crean las ramas en GitHub, en este caso debemos ponerlo aparte.

Por otro lado, el grupo decidió realizar una función operador, no lo para validar los datos ingresados, aunque, en clases concordamos que lo que se ingresa por parámetro, se toma como verificado, nos pareció conveniente para mostrar que el código funciona correctamente, sin problemas y por comodidad del docente al querer ver mejor todos los resultados ordenados en una lista con subsistas.

Justificación Fase 2

FASE 2: INCORPORACION DE LIBRERIA *LOCAL-TIME*.

Las librerías nos ayudan a simplificar el código, es un conjunto de archivos que se usan para desarrollar software suelen estar compuestas de una solución a un problema para después codificadas en un cierto lenguaje de programación específico son totalmente autónomas, y así ayudar a simplificar el desarrollo de los programas o brindas soluciones más sencillas, simplemente invocándolas.

Existen dos tipos distintos de librerías las estáticas y las dinámicas⁸ la primera se integra directamente al programa durante el proceso de compilación. Quiere decir que el código se copia dentro del archivo ejecutable, por lo que el programa puede seguir funcionando incluso si posteriormente se elimina la librería original, pero el tamaño del programa ejecutable suele ser mayor debido a que contiene una copia de todas las funciones utilizadas. Y la segunda, las librerías dinámicas, no suelen incorporarse al ejecutable durante la compilación. Lo que si hace es cargar en el programa las funciones necesarias en tiempo de ejecución. Permitiendo que varios programas compartan una misma librería y así el tamaño de los ejecutables se reduce, facilitando la actualización de funcionalidades. Empero, para que el programa funcione correctamente, la librería dinámica debe estar disponible en el sistema al momento de su ejecución.

La catedra en esta etapa no dio a elegir entre dos librerías *j.son* y *local-time* nos inclinamos a esta librería porque brindaba un formato para tiempo-unix, para la incorporación de la misma la página de Quicklisp nos ayudó a descargar el programa que funciona como un gestor de bibliotecas para Common Lisp, cuya se componen de bibliotecas con unos pocos comandos sencillos.

En nuestro grupo optamos por el uso de la librería “*local-time*” entre las 2 posibles opciones disponibles. Para la actividad se pedía modificar el sistema de auditoria perteneciente al Requerimiento 3 de la primera fase para que, en lugar de imprimir el tiempo en Formato-unix, la función retorna la fecha en un formato legible para humanos.

⁸ DevCamp. (s.f.). *Qué es una librería en programación*. DevCamp. Recuperado el 16 de junio de 2026, de <https://devcamp.es/que-es-libreria-programacion>

El primer paso para esto fue la instalación del archivo Quicklisp.lisp disponible en la página oficial. Seguido a esto, la instalación en el ecosistema de CLISP mediante las funciones que nos otorgaba la página. Una vez funcionando, investigamos sobre la librería local-time, su instalación y sus funciones,

En el caso de la biblioteca Local-Time su funcionamiento es más una biblioteca dinámica si bien nos ayudó a trabajar con fechas y horas en Common Lisp. La biblioteca debe encontrarse si o si instalada en la computadora para que el programa pueda acceder a sus funciones. Entonces cuando el programa se ejecuta, es necesario que tenga ruta de donde se instaló la biblioteca. Si la encuentra, se ejecuta y puede utilizar normalmente todas las funciones relacionadas con el manejo de fechas y horarios. Pero como nos pasó y esta explicado más adelante, si la biblioteca fue eliminada, movida de carpeta o no se encuentra instalada correctamente, el programa producirá el error Windows 32 y no la ejecutará. Un ejemplo de bibliotecas estáticas que aprendimos en nuestra investigación se puede encontrarse en el lenguaje de programación C, porque las funciones desarrolladas dentro de funciones como `#include<math.h>`, durante el proceso de compilación, el compilador copia al archivo ejecutable únicamente las funciones de la biblioteca que son necesarias para el programa. Por ejemplo, cuando nosotros utilizábamos funciones matemáticas como ``sqrt ()`` para calcular raíces cuadradas o ``pow ()`` para calcular potencias, enlazaba el programa con la biblioteca matemática de C, solo incorpora el código necesario antes de su ejecución. Esto contrasta con las bibliotecas dinámicas, que son cargadas por el programa recién cuando este se encuentra en funcionamiento.

Con la librería ya instalada pasamos a la modificación de la función original e implementación de las funciones propias de la librería para realizar la actividad.

```
(defun auditoria (tiempo-unix color-anterior color-nuevo)
  (format t "Tiempo ~a: la luz ha cambiado de ~a a ~a~%"
    (local-time:format-timestring nil
      (local-time:unix-to-timestamp tiempo-unix))
    color-anterior
    color-nuevo)
  )
```

Entonces comparándola lo que ingresamos por parámetro es similar, entonces lo que cambia es que los toma ese valor numérico y lo transforma en una cadena de fecha y hora como nosotros lo reconocemos por medio `local-time:unix-to-timestamp` que toma el tiempo número de tiempo-unix y `local-time:format-timestring` lo transforma en una cadena de fecha y hora el único problema es que el contador comienza desde 1 de enero de 1970 por lo que en los momentos de prueba tienen que ser con números muy extensos (auditoria 1700000000 'rojo 'verde) o podemos incorporar (auditoria (local-time:timestamp-to-unix (local-time:now)) "rojo" "verde") que también pertenece a la misma biblioteca, y está vinculada al reloj de la computadora.

Análisis Fase 3 y Preguntas teóricas Fase 3.

FASE 3: IMPLEMENTACION Y COMPARATIVA CON ERLANG.

Erlang es un lenguaje desarrollado en 1985 a 1997 por Joe Armstrong acompañado de colegas cuyo objetivo era encontrar mejores formas de programar sistemas de control de telecomunicaciones de gran escala⁹. A modo de ilustración las soluciones que propusieron fue crear sistemas telefónicos que deben formarse de por interrumpida, manejar un número extremadamente alto de actividades simultáneas, para que el software no pierda la estabilidad luego de un error crearon la filosofía "**let it crash**" (*deja que falle*), aislando los errores, entre otros.

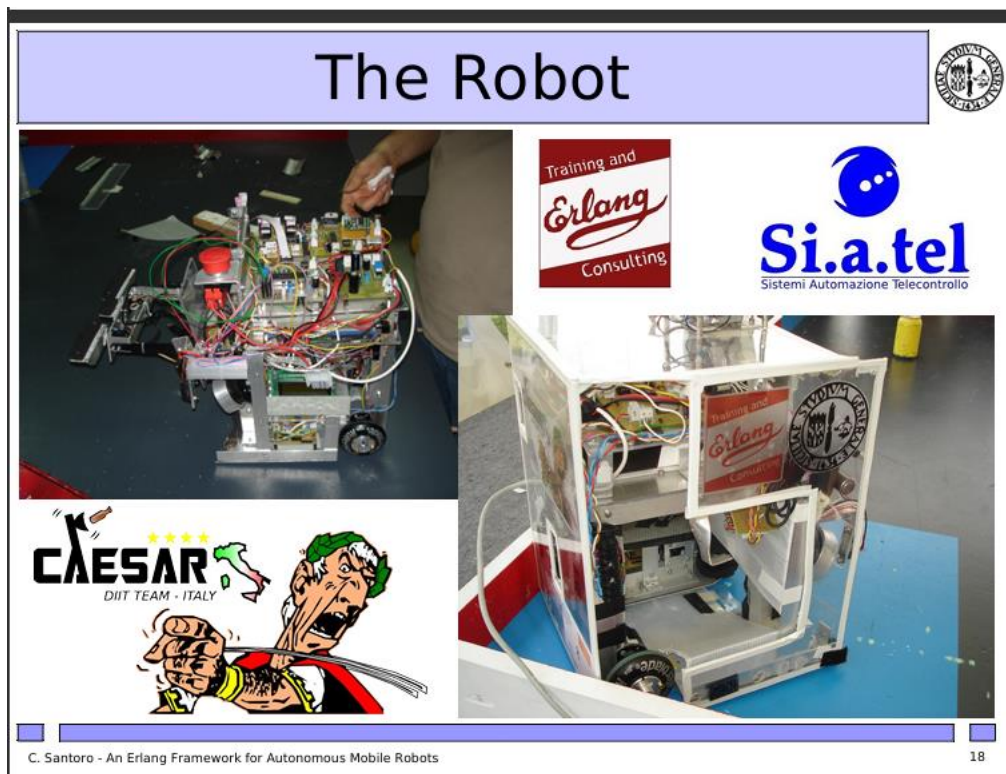
Los programadores concluyeron que los lenguajes declarativos permitían programas más cortos y claros, pero no tenían control de concurrencia y manejo de errores y Armstrong escribió un intérprete de Prolog, es decir un lenguaje declarativo el cual escribe el que debe resolver en vez del cómo, añadiendo conceptos de procesos y recuperación de errores evolucionando hasta el lenguaje que lo conocemos ahora.

Hoy en día Erlang es parte fundamental de los sistemas que requieren alta disponibilidad, estabilidad masiva y tolerancia a los fallos, Erlang es el motor de los sistemas de mensajería más grandes del mundo por manejar miles de conexiones simultáneas, por ejemplo, El servicio XMPP escrito en Erlang es utilizado para organizaciones como Jabber.org y fue la base para la infraestructura de Twitter, también T-mobile una compañía telefónica del Reino Unido lo utiliza para su gestión de mensajería.

De igual manera el sector financiero y Fintech, Kredito un banco en Suecia como Teba Bank en Sudáfrica el sistema es útil por el procesamiento de pagos, servicios financieros como la gestión de tarjetas de créditos, porque, el programa como hablamos anteriormente aísla los errores. De igual modo, también son parte de bases de datos por la gran cantidad de volúmenes de datos, otras áreas que se utiliza es en la inteligencia artificial del robot IANO, creado por los investigadores de la Universidad de Catania.

HOPL-III '07: ACM SIGPLAN History of Programming Languages Conference III. (2007).

ACM SIGPLAN. <https://dl.acm.org/doi/epdf/10.1145/1238844.1238850>



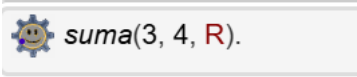
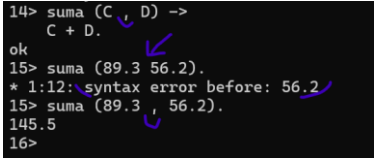
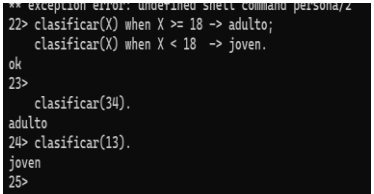
La estructura HW/SW del robot se muestra en la Figura (Santoro, 2007).

Por último, en los modelados 3D de los códigos abiertos de Wing 3D y, aunque no es su área principal, Erlang ha sorprendido en campos menos convencionales:

En resumen, Erlang se utiliza hoy principalmente en **software de infraestructura distribuida**, donde el costo de una caída del sistema es inaceptable y se requiere escalar linealmente con el hardware moderno.

Ahora bien, después de esta pequeña introducción nosotros para aprender a usar el nuevo lenguaje, seguimos las mismas instrucciones como los trabajos prácticos, es decir, primero escribimos variables, segundo comenzamos a realizar operaciones matemáticas sencillas, empezamos a escribir las primeras funciones con parametros, luego comenzamos a aumentar la complejidad con estructuras condicionales, a continuación, se ilustrará mejor las diferencias en la siguiente tabla.

Ejemplos	Prolog	Erlang
Variables	Las variables comienzan con mayúsculas, terminan con un punto y no puede asignar un nuevo valor sobre la misma. <pre> * 1.1. Syntax error before. * 10> A = 10. 10 11> a= 10. * exception error: no match of right hand side value 10 12> b=3. ** exception error: no match of right hand side value 3 13> </pre>	
Funciones	En prolog podemos llegar a decir que es similar en algunos aspectos para empezar el nombre de la función es	Las funciones comienzan con el nombre en minúscula porque como vimos

	en minúscula y las variables en mayúsculas, sin embargo, no depende de la flecha si no que se suplanta por “: “y “-”, por añadiría el tercer elemento nos ayuda a brindar el resultado al no ser un lenguaje imperativo si no lógico es más que necesario, porque genera la pregunta ¿hay un valor en resultado que haga verdadera la suma de X + Y? Y eso lo imprime, porque encontró un valor verdadero. <pre>suma(X, Y, Resultado) :- Resultado is X + Y.</pre>  R = 7	anteriormente solo las variables comienzan con mayúsculas. Seguido de los paréntesis que incluyen las variables separadas por cómo y una fecha que invoca a la lógica de predicados, la implicación, condición que <i>si p -> “entonces” q</i> donde p es el antecedente y q el consecuente. Esto se aplica de la siguiente manera si p= si llamo a la función suma entonces q= suma las variables C y D. Seguidamente se llama a la función con el nombre y los parametros deseados, como vemos a quien si no los separamos con cómo nos brinda la imagen de error de sintaxis. 
Los puntos y comas	En Prolog los puntos y comas nos sirven para llamar el operador lógico OR, <pre>es_adulto_o_joven(X) :- (X >= 18 -> write('Es adulto') ; write('Es joven')).</pre>  Es adulto true <pre>?- es_adulto_o_joven(25)</pre>	En Erlang nos sirve para separar los distintos tipos de cuestiones a evaluar en una función además, demuestra que pertenece a esa misma, y la función termina con un punto al final. 

Ahora en el caso de lips, en la primera función se utilizó el método *pattern matching*

O coincidencia de patrones que es la base estructural donde el programa unifica el control del flujo ligado a varias opciones. Bueno como ya aprendimos la asignación de Variables en General siempre son de un solo valor por lo que, como en erlang no existe la función “equal” explícitamente, una forma que encontramos la solución fue llamar átomos como parametros, a diferencia de lips, aquí un átomo es una constante cuyo valor es su propio nombre no cambia ni altera su contenido , por lo tanto, si es rojo será su valor rojo comienzan con minúsculas porque no es una variable ya que solamente evalúa si es igual o no a ese nombre , porque si le prosigue = lo evaluaría y guardaría un valor que aquí no queríamos hacer eso.

Luego de verificar que es igual a cualquiera de ‘EnVerde ‘en Amarillo ‘rojo imprime el resultado como si fuera una lista, acá a diferencia de lips las listas no se rigen por paréntesis sino por [], [en_amarillo, cambiar_a_rojo]. Y si no es ninguno copia el nombre ingresado por parámetro en una Variable que pasa como argumento lo copia y los imprime, a diferencia de las variables que han sido unificada con un valor por un “=” y no puede cambiar, esta es indefinida y puede asociarse a cualquier valor.

Otra diferencia que podemos ver es como llamamos la función, el Lips se invoca con un paréntesis seguido de defun (defun Nombre función ()), cuyo nombre no tiene diferencia ya que Lips al ejecutar el programa pasa todas las variables a mayúsculas, en cambio en Erlang el nombre de la función debe ser si o si en minúscula como vimos anteriormente seguida de los parametros y la -> en caso de que haya varias condiciones es recomendable colocar; y cuando finaliza. En cambio, en Lips cada paréntesis que se cierra tiene que ver con una cierta función, **((and (equal color-actual 'en-verde) (equal cambiar-a 'amarillo))**

(List color-actual 'cambiar-a-amarillo)) Por ejemplo aquí este paréntesis corresponde a la evaluación de and, (equal color-actual 'en-verde) y este al ver si el argumento pasado por parámetro es igual a la palabra en verde y así sucesivamente.

En Erlang

```
49> transicion_semf(en_rojo, verde) ->
      [en_rojo, cambiar_a_verde];
      transicion_semf(en_verde, amarillo) ->
      [en_verde, cambiar_a_amarillo];
      transicion_semf(en_amarillo, rojo) ->
      [en_amarillo, cambiar_a_rojo];
      transicion_semf(ColorActual, _) ->
      [ColorActual, transicion_invalida].
ok
50> transicion_semf(enRojo, verde).
[enRojo,transicion_invalida]
51> transicion_semf(en_verde, amarillo).
[en_verde,cambiar_a_amarillo]
52> transicion_semf(en_amarillo, rojo).
[en_amarillo,cambiar_a_rojo]
53> transicion_semf(en_amarillo, en_rosa).
[en_amarillo,transicion_invalida]
54>
```

En Lips

```
Break 1 [3]> ;; FUNCIÓN: transicion-semf
;; NATURALEZA: Pura (dado un timestep, siempre retorna el mismo color)
;; ESTRATEGIA: Selectiva (uso de cond para elegir una transición)
;; IMPACTO: No destructiva (solo calcula y devuelve resultados)
;; =====
(defun transicion-Semf (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'en-verde)(equal cambiar-a 'amarillo))
      (list color-actual 'cambiar-a-amarillo))
    ((and (equal color-actual 'en-amarillo)(equal cambiar-a 'rojo))
      (list color-actual 'cambiar-a-rojo))
    ((and (equal color-actual 'en-rojo)(equal cambiar-a 'verde))
      (list color-actual 'cambiar-a-verde))
    (t (list color-actual 'transicion_invalida)))
  )
)

TRANSICION-SEMF
Break 1 [3]> (transicion-Semf 'en-verde 'amarillo)

(EN-VERDE CAMBIAR-A-AMARILLO)
Break 1 [3]> (transicion-Semf 'en-amarillo 'rojo)

(EN-AMARILLO CAMBIAR-A-ROJO)
Break 1 [3]> (transicion-Semf 'en-rojo 'verde)

(EN-ROJO CAMBIAR-A-VERDE)
Break 1 [3]> (transicion-Semf 'en-rojo 'rosa)

(EN-ROJO TRANSICION_INVALIDA)
Break 1 [3]> |
```

En caso de no poder ver las imágenes sugerimos hacer zoom para apreciar mejor la diferencia.

En cambio, en tiempo unix, al no tener que comparar un nombre igual si no que debe evaluar un valor numérico basado en los segundos que se ingresa, cambia la forma en cual se realiza dentro de la función, porque evalúa el valor del resto de la división entre los seg ingresado y el ciclo total de 216 seg del semaforo. En el caso que sea menor a los 90 seg. Imprimirá el mensaje

“EN ROJO” así sucesivamente como hicimos la función en lips. Lo diferente es la implementación del **when**, permitiéndonos comprar con operadores lógicos, pero no acepta una evaluación comparando un contenido de una función porque no garantiza nada sobre ella porque erlang no sabe qué hace la otra función dentro porque when está diseñado para ser seguro y predecible.

En Erlang

```
54> timer (TiempoUnix) when TiempoUnix < 0 -> "ERROR: LOS SEGUNDOS NO PUEDEN SER NEGATIVOS";
    timer (TiempoUnix) when TiempoUnix rem 216 < 90 -> "EN ROJO";
    timer(TiempoUnix) when TiempoUnix rem 216 < 96 -> "EN AMARILLO";
    timer (TiempoUnix)-> "EN VERDE".
ok
55> timer(-3).
"ERROR: LOS SEGUNDOS NO PUEDEN SER NEGATIVOS"
56> timer(23).
"EN ROJO"
57> timer(93).
"EN AMARILLO"
58> timer(120).
"EN VERDE"
59> timer(434).
"EN ROJO"
60>
```

En Lips

```
EN-ROJO
Break 6 [8]> ;; =====
;; FUNCIÓN: timer
;; NATURALEZA: Pura (dado un timestamp, siempre retorna el mismo color)
;; ESTRATEGIA: Selectiva (uso de cond)
;; IMPACTO: No destructiva (solo calcula y devuelve resultados)
;; =====

(defun timer (tiempo-unix)
  (cond
    ((< tiempo-unix 0)
     "El tiempo Unix no puede ser negativo")
    ((rem tiempo-unix (sumar-ciclos)) 90)
     'en-rojo)
    ((rem tiempo-unix (sumar-ciclos)) 96)
     'en-amarillo)
    (t
     'en-verde)))

TIMER
Break 6 [8]> ;; =====
;; FUNCIÓN: Sumar-ciclos
;; NATURALEZA: Pura.
;; ESTRATEGIA: Devuelve una suma de la cant. de segundos de duración del semf. en rojo 90 seg., amarillo 6 seg. y verde 120 seg.
;; IMPACTO: No destructiva(Auxiliar para funcion timer y recomendacion-ciclo-ing)
;; =====

(defun sumar-ciclos ()
  (+ 90 6 120))
```

Sin embargo, después de la nueva corrección de timer y al agregar nuevos elementos a evaluar decidimos que un if como estructura condicional cumple la misma función en erlang pero, if a diferencia de when, evita la redundancia en la cabecera, porque así recibe el argumento una vez y la lógica se ramifica directamente, porque son parte de una única decisión basada en el

tiempo unix. además, acá hay un manejo explícito del else, con true-> "amarillo_intermitente". Por otra parte, ambos son condiciones que evalúan cada opción como si fuera un cond puede evaluar hasta encontrar el primer true o hasta que llega al final y Tenes una condición t. A diferencia de lips acá no se puede llamar a otras funciones porque erlang garantiza que evaluar la condición no tenga efectos secundarios.

```
timer(TiempoUnix) when TiempoUnix < 0 ->
```

```
    "ERROR: LOS SEGUNDOS NO PUEDEN SER NEGATIVOS";
```

```
timer(TiempoUnix) ->
```

```
    if
```

```
        (TiempoUnix rem 225) < 90 -> en_rojo;
```

```
        (TiempoUnix rem 225) < 93 -> rojo_intermitente;
```

```
        (TiempoUnix rem 225) < 213 -> en_verde;
```

```
        (TiempoUnix rem 225) < 216 -> verde_intermitente;
```

```
        (TiempoUnix rem 225) < 222 -> en_amarillo;
```

```
        true -> amarillo_intermitente
```

```
    End.
```

Además, a ser un elemento numérico a diferencia de la función anterior, con ello permite primer ver todas las decisiones a evaluar, solo recibe el argumento una vez, devuelve sino un true respetando la filosofía "let it crash" (deja que falle) y el *end* muestra donde termina la función.

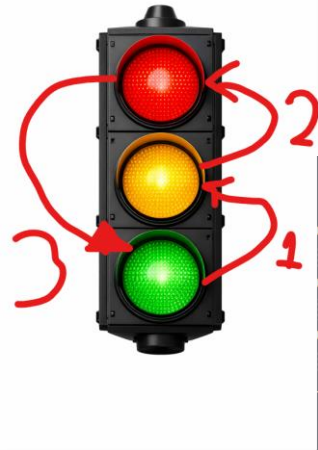
En resumen, decimos que el pensamiento lógico de resolver el problema es exactamente lo mismo porque ambas son parte del paradigma funcional en el que se basa la resolución de problemas mediante la función matemática de composición de funciones, que devuelven un solo valor, dado el parámetro o la lista de parametros. Tanto Erlang como Lisp comparten esta forma de pensar, ya que utilizan funciones y recursión para resolver problemas lógicos. Sin embargo, la sintaxis de cada lenguaje es diferente. Erlang se apoya principalmente en el pattern matching, la inmutabilidad de las variables y la recursión estricta. Por su parte, Lisp trabaja listas, utilizando funciones como elemento central del lenguaje. Aunque la forma de escribir el código cambia, la lógica de resolución de problemas sigue siendo la misma porque ambos pertenecen al paradigma funcional.

BITACORA DE ERRORES.

Errores fase 1

Creeríamos nosotros que esta es una parte importante del proceso de resolución porque tuvimos muchos errores, modificaciones no solamente por la resolución del código si no, por la forma de pensar los problemas ,las interpretaciones e incluso el aprendizaje autónomo de nuevas cosas como el repositorio, la instalación de la librería y el aprendizaje de un lenguaje nuevo como Erlang, si bien es sabido, que como futuro programadores debemos resolver problemas las dudas, desesperanzas y trabas no se hicieron esperar Está en. Cuando nosotros estuvimos haciendo este informe nos dimos a los otros errores a la hora de explicarlo. Entonces, como se va a ver en el repositorio, tenemos mucho Commits en el repositorio porque fuimos observando, cambiando nombres, forma de pensar nuevas soluciones, cuestionamientos, etc.

En primer lugar, la mala interpretación del ciclo del semáforo si nosotros ilustramos la primera secuencia nos quedaría así: **1.** El verde iría al amarillo **2.** El amarillo al rojo **3.** Del rojo al verde directamente saltándose. Directamente el amarillo, siendo erróneo y caótico en la vida real. Cuando lo correcto era hacer del **1.** Rojo al verde, **2.** Verde a amarillo **3.** Del amarillo al rojo.



El código anterior quedo así:

```
transicion (color-actual cambiar-a)
  (list color-actual 'cambiar-a-amarillo))
  (list color-actual 'cambiar-a-rojo))
  (list color-actual 'cambiar-a-verde))
  (list color-actual 'accion-por-defecto))
```

Otro error que tuvimos fue, que al desarrollar esta función no habíamos puesto el Quote al comparar el parámetro “color-actual” y “cambiar-a”, ya que sin ese Quote lo interpreta como una variable y no como el símbolo, también debatimos si usar la función EQ o EQUAL y nos decidimos por el EQ ya que comparamos símbolos.

Después en Tiempo-Unix, primero se creó con let para asignar un valor numérico a segundos, al evaluarlas todos juntos llegamos a la conclusión que esta forma de resolver es errónea porque no puede tener mantenibilidad en la cantidad de los 216 segundos, además let utilizamos para el ingreso de variables locales en los operadores y sería más del paradigma imperativo que el funcional.

```

Break 5 [7]>
(defun timer (tiempo-unix)
  (let ((segundo (rem tiempo-unix 216)))
    (cond
      ((and (>= segundo 0) (<= segundo 89)) 'en-rojo)
      ((and (>= segundo 90) (<= segundo 95)) 'en-amarillo)
      ((and (>= segundo 96) (<= segundo 215)) 'en-verde)
    )
  )
)

```

Una de las formas que usamos para arreglar ello fue crear una composición de funciones con la suma del ciclo, en un principio pensamos que si habría que darle mantenibilidad no tendría que cambiar el ciclo total, si no los seg. De cada color por separado, además que si hacemos así por separado la función cada función tiene una única responsabilidad, que al final con la ultima correccion, es errónea, ya que el ciclo se incorpora los semaforos con interminencias, mas 3 segundos de cada intermitencia.

```

Break 6 [8]> ;; =====
;; FUNCIÓN: Sumar-ciclos
;; NATURALEZA: Pura.
;; ESTRATEGIA: Devuelve una suma de la cant. de segundos de duración del semf. en rojo 90 seg., amarillo 6 seg. y verde 120 seg.
;; IMPACTO: No destructiva(Auxiliar para función timer y recomendación-ciclo-ing)
;; =====
(defun sumar-ciclos ()
  (+ 90 6 120))

```

En el requerimiento número 4a. los problemas no se hicieron esperar creo que de todos fue el requerimiento con la redacción más confusa y con más cambio que hicimos en un principio,

porque primero confundimos el punto 4a con el 4b e hicimos al revés, después pensamos la forma de resolver el problema era como hicimos en el requerimiento 5 porque a lo que nosotros nos confundía era esto ¿cómo va a ser un solo número? Si tienes que calcular la duración total y no especifica si en minutos, en horas, es días, entonces decidimos que el cálculo es un ingreso por parametro de las duración del semaforo en rojo, en amarillo y en verde. Primero porque pensamos que el usuario tiene que probar la duración del ciclo total por color de semaforo.

- **Entrada:** un número determinado de segundos

```

Break 7 [9]> (defun sumar-ciclos ()
              (+ 90 6 120))
(defun duracion-ciclo (nro-ciclos)
  (truncate (float (/ nro-ciclos (sumar-ciclos))))

SUMAR-CICLOS
Break 7 [9]>
DURACION-CICLO
Break 7 [9]>

```

- **Propósito:** *Calcular duración total de un ciclo completo (rojo → amarillo → verde → rojo)*

Una vez superado esto solo tuvimos problemas de sintaxis pequeños como, fue el ($\leq$ nro-segundos 35) en la primera opción del cond tuvimos el pequeño error de decir que sea menor o igual, empero, en los requerimientos proponía que el rango optimo es desde 35 hasta 150 segundos por los que tuvimos que cambiarlo.

```
DURACION-CICLO
Break 8 [12]> (defun duracion-ciclo (nro-segundos)
  (cond
    ((<= nro-segundos 35) "Ciclo de duracion bajo. Ajustar a optimo")
    ((<= nro-segundos 150) "Ciclo de duracion optimo")
    (t "Ciclo de duracion alto")))

DURACION-CICLO
Break 8 [12]> (duracion-ciclo 23)

"Ciclo de duracion bajo. Ajustar a optimo"
Break 8 [12]> (duracion-ciclo 230)

"Ciclo de duracion alto"
Break 8 [12]> (duracion-ciclo 130)

"Ciclo de duracion optimo"
Break 8 [12]> (duracion-ciclo 35)

"Ciclo de duracion bajo. Ajustar a optimo"
```

Otro error fue que al realizar la composición de funciones no entendemos cuál era el error y este fue el olvido de escribir el argumento dentro de “calcular-ciclos”.

```
Break 8 [12]> (defun recomendacion-ciclo-ing (sem-R sem-A sem-V)
  (duracion-ciclo (calcular-ciclos)))

RECOMENDACION-CICLO-ING
Break 8 [12]> (recomendacion-ciclo-ing 34 23 120)

*** - EVAL/APPLY: Too few arguments (0 instead of at least 3) given to CALCULAR-CICLOS
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
ABORT      :R7      Abort debug loop
ABORT      :R8      Abort debug loop
ABORT      :R9      Abort main loop
Break 9 [13]> (calcular-ciclos 34 23 120)

177
```

En el requerimiento 5 al ya tener la idea lógica plasmada, no tuvimos problema alguno es hacer porque ya este hecho y solo cambiamos algunas cosas.

En el requerimiento 6 nos había pasado igual que este Cambiamos la función "calcular- Porcentaje-Horas" ya que al utilizar en un principio el let creaba una variable que guardaba un valor temporal, siendo ella más propio del paradigma imperativo. Al evitar ello remplazamos el let con la función " calcular-ciclos" ya existente, logrando la reutilización de código y composición funcional. Otra vez en una tercera revisión nos dimos cuenta de que tenía otro problema, la función decía por horas, pero acá no se contemplan esa cantidad de segundo que completan una hora.

```

Break 7 [9]> (defun calculaPorcentajesHora1 (semf-R semf-A semf-V)
  (list
    'ROJO (float (* (/ rojo (calcular-ciclos semf-R semf-A semf-V)) 100))
    'AMARILLO (float (* (/ amarillo (calcular-ciclos semf-R semf-A semf-V)) 100))
    'VERDE (float (* (/ verde (calcular-ciclos semf-R semf-A semf-V)) 100))))

CALCULAPORCENTAJESHORA1
Break 7 [9]>

```

Nuevamente luego hicimos de forma errónea porque ahí sacaba un porcentaje en base al ciclo, no en base a una hora, aunque el porcentaje no cambie sin importar la duración, decidimos agregar la conversión de segundos a horas.

En la parte iterativa tuvimos los errores como olvidar que with-open-file cierra el archivo automáticamente.

Otro error que print como hacíamos de costumbre en lugar de format stream, pero nos hizo recordar Lips que print escribe en la consola y no en el archivo, otro error fue asimilar que porque es iterativa la función debemos agregar do-list para el informe porque pensamos que la iteración sobre los datos con 'dolist' nos sirve para procesar la lista de registro porque asigna una variable local dentro del registro a cada elemento de la lista y ejecuta la función hasta que no queden más elementos. Por otra parte, en el mismo bucle pensamos que para extraer los componentes de cada lista utilicemos (**car registro**): para extraer el primer elemento, que corresponde a la **fecha y hora** (Timestamp). Después el (**cadr registro**) extrae el segundo elemento, verbigracia el **estado de origen** (ej. ROJO) y ergo, (**caddr registro**), extrae el tercer elemento: el **estado de destino** (ej. VERDE). Con este formato "~A - Transición: ~A -> ~A~%", por lo tanto, cada llamado dentro del dolist añade una nueva línea al archivo hasta completar el historial de movimientos del semáforo

Recapitulando esta función transforma una estructura de datos en un **documento de texto** cumpliendo con las reglas de estilo de la cátedra al utilizar una descomposición clara y herramientas de iteración adecuadas como dolist en lugar de recursión manual para tareas de salida de datos.

```
(defun informe (datos)
```

```
  (with-open-file (stream
```

```
    "informe-ejecucion-semaforo.txt"
```

```
    :direction :output
```

```
    :if-exists :supersede)
```

```
    (format stream "Informe de Ejecución del Sistema Semafórico~%")
```

```
(format stream "=====~%~%")
```

```
(dolist (registro datos)
```

```
  (format stream "~A - Transición: ~A -> ~A~%"
```

```
    (car registro)
```

```
    (cadr registro)
```

```
    (caddr registro)))
```

```
(format stream "~%--- Fin del Informe ---")))
```

Por ejemplo:

Informe de Ejecución del Sistema Semafórico

```
=====
```

2024-06-04 14:30:15 - Transición: ROJO -> VERDE

2024-06-04 14:30:30 - Transición: VERDE -> AMARILLO

2024-06-04 14:30:35 - Transición: AMARILLO -> ROJO

Errores Fase 2.

En el punto de la incorporación de la librería en la fase 2, la problemática mayoritaria fue más la instalación de la biblioteca dentro del intérprete sublime, que inclusive la realización del código. Nuestro problema más grande es que una de las compañeras no lo pudo instalar en ese sublime, ya que el programa les lanzaba mensajes de error. Sin embargo, uno de nuestros compañeros pudo instalarlo con muchas complicaciones como su primer inconveniente surgió durante la implementación de quicklisp en CLISP. Arrojo varios errores con la función (load) para cargar el archivo.

- (Load "~/quicklisp/setup.lisp")

Arrojaba errores como Error de Win32 267 (ERROR_DIRECTORY) a pesar de que el archivo exista y verificar el directorio correcto. Llegamos a la conclusión de que la función load no funcionaba correctamente y buscamos otras alternativas en otras versiones de Lisp, lo cual no

tuvo resultados. Por lo tanto, decidimos cambiar de intérprete a SBCL (Steel Bank Common Lisp) el cual tiene un funcionamiento muy similar a CLISP e implementada también con Common Lisp. Luego no hubo más problemas con la instalación de los otros paquetes.

Errores Fase 3.

En esta parte los errores de lógica no fueron tantos, no porque sepamos de antemano o hayamos programado con Erlang con anterioridad, si no que al tener el conocimiento del paradigma funcional de la catedra ,más que para acostúmbrenos a la nueva sintaxis lo que hicimos fue seguir lo lineamientos que nos dio los trabajos prácticos, al principio no vamos a negar que tuvimos muchas dudas, pero seguir ese camino fue correcto entonces, la ayuda de las tutorías de las inteligencias artificiales como Claude nos ayudaron a entender mejor desde la implementación de una variable simple hasta la creación de funciones. Como podemos ver este fue nuestro camino para llegar a realizar las funciones de una manera más clara.

Después al incorporar tuvimos muchísimos errores al manejar las funciones para no ilustrar con una imagen tan grande, escribiré aquí todo,

```
15> sumarCiclos()-> 90 + 6 + 120.
```

```
ok
```

```
16> timer (TiempoUnix) when TiempoUnix rem  
sumarCiclos < 90 -> "EN ROJO";
```

```
    timer(TiempoUnix) when TiempoUnix rem  
sumarCiclos < 96 -> "EN AMARILLO";
```

```
    timer(TiempoUnix)-> "EN VERDE".
```

```
ok
```

```
17> timer(300).
```

```
"EN VERDE"
```

```
18> timer(87).
```

```
"EN VERDE"
```

```
19> timer(2).
```

```
"EN VERDE"
```

```
20> timer (TiempoUnix) when TiempoUnix rem sumarCiclos() < 90 -> "EN ROJO";
```

```
    timer(TiempoUnix) when TiempoUnix rem sumarCiclos() < 96 -> "EN AMARILLO";
```

```
    timer(TiempoUnix)-> "EN VERDE".
```

```
* 3:44: illegal guard expression
```

```
21> timer (TiempoUnix) when TiempoUnix rem sumarCiclos() < 90 -> "EN ROJO";
```

```
    timer(TiempoUnix) when TiempoUnix rem sumarCiclos() < 96 -> "EN AMARILLO";
```

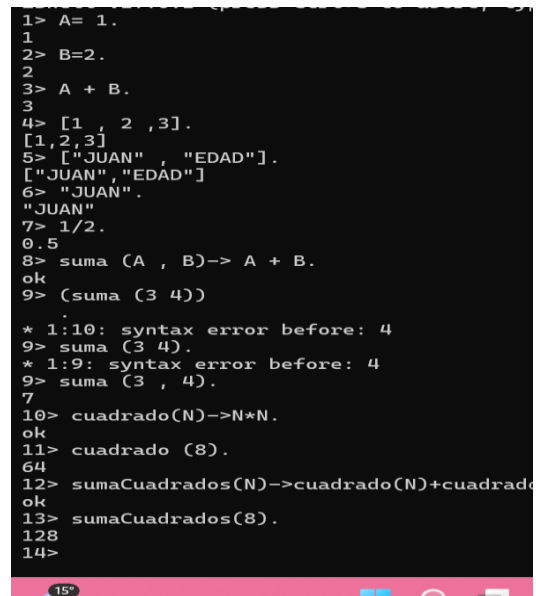
```
    timer(TiempoUnix)-> "EN VERDE".
```

```
* 3:44: illegal guard expression
```

```
22> timer1 (TiempoUnix) when TiempoUnix rem sumarCiclos() < 90 -> "EN ROJO";
```

```
    timer1(TiempoUnix) when TiempoUnix rem sumarCiclos() < 96 -> "EN  
AMARILLO";
```

```
    timer1(TiempoUnix)-> "EN VERDE".
```



```
1> A= 1.  
1  
2> B=2.  
2  
3> A + B.  
3  
4> [1, 2, 3].  
[1, 2, 3]  
5> ["JUAN", "EDAD"].  
["JUAN", "EDAD"]  
6> "JUAN".  
"JUAN"  
7> 1/2.  
0.5  
8> suma (A , B)-> A + B.  
ok  
9> (suma (3 4))  
.   
* 1:10: syntax error before: 4  
9> suma (3 4).  
* 1:9: syntax error before: 4  
9> suma (3 , 4).  
7  
10> cuadrado(N)->N*N.  
ok  
11> cuadrado (8).  
64  
12> sumaCuadrados(N)->cuadrado(N)+cuadrado(N)  
ok  
13> sumaCuadrados(8).  
128  
14>
```

```

* 3:44: illegal guard expression
23> Timer2 (tiempoUnix) when TiempoUnix rem 216 < 90 -> "EN ROJO";
    timer3(tiempoUnix) when TiempoUnix rem 216 < 96 -> "EN AMARILLO";
    tiemer3 (tiempoUnix)-> "EN VERDE".
* 1:21: syntax error before: 'when'
23> timer4 (TiempoUnix) when TiempoUnix rem 216 < 90 -> "EN ROJO";
    timer4(TiempoUnix) when TiempoUnix rem 216 < 96 -> "EN AMARILLO";
    timer4 (TiempoUnix)-> "EN VERDE".
ok
24> timer4(300).
"EN ROJO"
25> timer4(6723).
"EN ROJO"
26> Semaforo(87).
* 1:1: variable 'Semaforo' is unbound
27> timer(87).
"EN VERDE"
28> timer(93).
"EN VERDE"
29> timer4(2).
"EN ROJO"
30> timer4(93).
"EN AMARILLO"
31>

```

Cuyo problema radicaba en que ‘when’ no acepta como explicamos anteriormente, funciones si no operadores lógicos con comparación de elementos numéricos.

Otro de los errores, pero más miedos es el uso del repositorio GitHub, al no estar familiarizados con la plataforma que la misma este en inglés, siendo para nosotros menos intuitiva, hicimos que un integrante del grupo se haga cargo de la investigación para explicar a los demás, si bien no falta muchísimo entenderla, creo que para ser un comienzo aprendimos lo básico cuando hay muchas cosas buenas para sacar jugo en esta página.

También hicimos un cronograma de trabajo <https://canva.link/uuwn259v6xm8uu7>, tratamos de juntarnos y poder llegar con los objetivos, incluimos tiempos que se puede perder y si nos pasó porque o no nos podíamos juntar por las actividades diarias o también porque teníamos que rendir parciales, pero en general siempre buscamos juntarnos y aclarar los objetivos.

Por último, creo que lo que más nos costó fueron la redacción confusa de los requisitos era poco explícito y a veces con errores lógicos que hacían que la imaginar la simulación se vuelva poco clara, tuvimos que discutir con nuestros compañeros de otros grupos y pensarlos horas, aunque si es una simulación de la vida real de un semaforo pensamos que el día de mañana muchos clientes nos vendrán con ideas vagas y sin sentido y queda en nosotros interpretarlo.

Conclusión.

A lo largo del extenso trabajo, podemos concluir que dentro de la clasificación de semáforos propio de la distinción que realizan los ingenieros civiles, nuestro programa se asemeja a un semáforo de un tiempo fijo porque las fases del ciclo tienen ya una duración establecida con 216 segundos. Ya definidos en los requisitos y en este caso se ejecuta siempre en el mismo orden y cantidad de segundos y los estados intermitentes son siempre los mismos.

En la vida real según el tráfico y caudal de personas hay semáforos accionados por el tráfico que utilizan sensores o detectores para medir la cantidad de vehículos en la intersección y a partir de ello va adaptándose a las condiciones reales o los semáforos de control centralizados que reciben información de distintas intersecciones y toma decisiones para optimizar el tránsito en las diferentes áreas de la ciudad, que son utilizados en grandes áreas urbanas, empero los requerimientos de nuestro semaforo nos impidió la simulación de ello.

En lo que concierne como aprendizaje grupal el aprendizaje y la resolución del código no fue lineal y estuvo lleno de errores, incertidumbres, pero mediante eso aprendimos a comprender mejor los requerimientos al Tenes que pensar bastante, preguntarnos que esperábamos que haga, leyendo una y otra vez y como esto se refleja en los commics en GitHub.

Uno de los más importantes fue la discusión de la interpretación del orden de los semáforos que discutíamos con nuestros compañeros de otros grupos para entender mejor luego de la corrección en la reunión por meet y haberlo plasmado en una imagen logramos implementarlo, otro aprendizaje fue pasar de las soluciones con let y pasar a variables temporales, cuando en realidad después de cada reunión con el grupo buscamos que las funciones tengan una única responsabilidad mediante la composición de funciones, evitando problemas, como paso con la función calcularPorcentajesHora, que reutiliza calcular-ciclos en lugar de duplicar la suma, fue el momento en el que sentimos que algo había cambiado en nuestra manera de pensar. Después la incorporación de la librería local-time nos enfrentó a un desafío inesperado, primero el de la instalación, y las dependencias para abrir en la librería en el lugar indicado, y ahí aprendimos que programar no es solo escribir código aislado, sino también entender el ecosistema, leer documentación, descargar interpretes como sbcl y resolver problemas que pasan con cada fase, superando frustraciones y enojos.

Y el desafío de pasar a Lips a Erlang, desde un principio, no nos acostumbrábamos a la sintaxis si bien la esencia es la misma la influencia de Prlog nos confundió al principio los átomos en minúscula, los puntos y comas con significados tan precisos. Sin embargo, resolvimos el problema yendo de apoco, se mencionó la famosa frase del filoso Epíteto, ***"Es imposible empezar a aprender lo que uno cree que ya sabe."*** Con ello querer aplicar el conocimiento de la sintaxis de Lips era en vano comenzando con los mismo paso pero en Erlang y tardamos muchos menos en aprender que si queríamos hacer la función sola, esto nos hizo entender que el conocimiento del paradigma se transfiere de un lenguaje a otro nos dio un poco más de confianza, llegando a mostrar que los errores como aquel el uso de when que ya fueron errores mínimos que nos enseñaron sutilezas sobre la evaluación en Erlang que no habríamos notado de otro modo.

Por último, la bitácora de errores que incluimos todo nuestro proceso, que incluso fuimos corrigiendo durante la realización del informe, debido a que cuando uno quiere explicar a otro con sus palabras ahí se dé cuenta en verdad que está mal y que no, Por lo tanto, cada error de sintaxis, cada mala interpretación del ciclo, cada función que tuvimos que reescribir para hacerla más pura o reutilizable, fue un escalón, que como grupo nos acompañamos.

Bibliografía

- **Autor desconocido. (s.f.).** Cálculo de tiempos en semáforos. Scribd. Recuperado de <https://es.scribd.com/presentation/435479012/calculo-de-tiempos-en-semaforos>
- **Agencia Nacional de Seguridad Vial. (2008).** *Manual del conductor para el curso de seguridad vial*. Ministerio del Interior de la Nación, República Argentina. https://www.argentina.gob.ar/sites/default/files/ansv_licencias_manual_del_conductor.pdf
- **Congreso de la Nación Argentina. (1995).** *Ley Nacional de Tránsito N. ° 24.449*. Texto actualizado. Artículo 44 – Vías semaforizadas. Recuperado de <https://www.argentina.gob.ar/normativa/nacional/ley-24449-818/actualizacion>
- **Armstrong, J. (1997).** *The development of Erlang*. Computer Science Laboratory, Ericsson Telecom AB.
- **Armstrong, J. (s. f.).** *A History of Erlang* [Diapositivas de presentación]. HOPL-III, Ericsson AB.
- **Hébert, F. (s. f.).** *Higher Order Functions*. Learn You Some Erlang for Great Good!
- **HOPL-III '07: ACM SIGPLAN History of Programming Languages Conference III.** (2007). ACM SIGPLAN. <https://dl.acm.org/doi/epdf/10.1145/1238844.1238850>
- **Monzón, R. (2026).** *PyL - 2026 - Introducción* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.
- **Monzón, R. (2026).** *PyL - 2026 - Concurrency y Paralelismo 1* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.
- **Monzón, R. (2026).** *PyL - 2026 - Concurrency y Paralelismo 2: Descomposición de Problemas y Especificación Formal* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.
- **Monzón, R. (2026).** *PyL - 2026 - Funcional Parte 2: Funciones para operar sobre listas y asignación* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.
- **Monzón, R. (2026).** *PyL - 2026 - Funcional Parte 3: Estructuras condicionales, iterativas y entrada/salida* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.

- **Monzón, R. (2026).** *PyL - 2026 - Funcional Parte 4: Asignación, funciones lambda, recursividad y streams* [Diapositivas de PowerPoint]. Paradigmas y Lenguajes, Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste
- **Unknown. (2014, abril 21).** Distribución de los tiempos de semáforos (I). Todo de Ingeniería Civil. Recuperado de Unknown. (2014, abril 21). Distribución de los tiempos de semáforos (I). Todo de Ingeniería Civil.